HKU BUSINESS SCHOOL
港大經管學院

Git and GitHub

Lecture 4 Extension

ECON6067: Quantitative Tools for Economic Analysis: From Math to AI

Yan Xiong

Which version of your skill produced the result you checked?

Two Parts, One Research Workflow

Part I: Git

Record and inspect versions on your computer.

Recover from a bad change and understand branches.

Practice: a baseline commit and a checked revision.

Part II: GitHub

Read repositories and publish your project online.

Follow a live website demo of publishing, access, and review.

Your project: share a private repository with the teaching team for grading.

Track your inspect-data skill; explore CC Switch as an open-source example.

Part I: Git

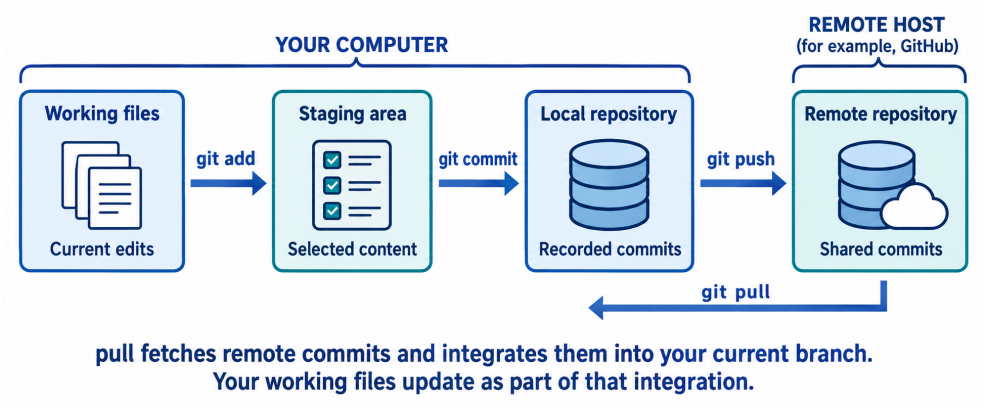
Version control for your project files

Git records selected versions so you can inspect how a project changed.

1. **Four areas and commands**: where your changes go.
2. **Branches**: separate lines of development.
3. **Git in AI-assisted work**: review, checkpoints, and recovery.

You can make local commits without a GitHub account.

1. Four Areas and the Git Commands



`git commit` records the staged snapshot as a new commit in the local repository.

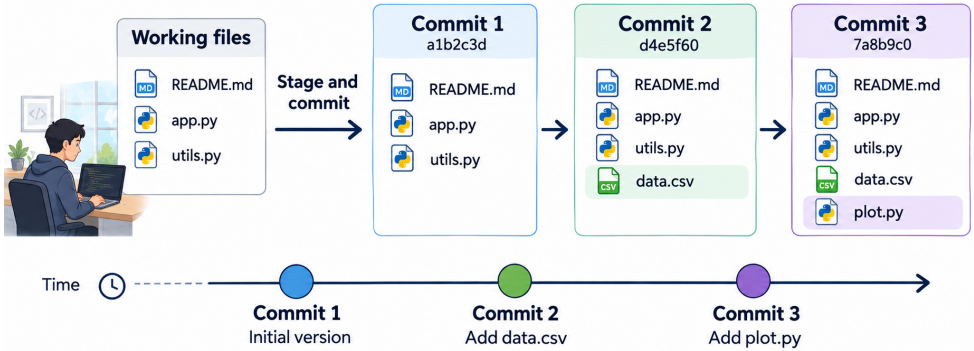
The Commands Behind the Arrows

Command	Action
`git add <file>`	Select this file's current content for the next commit.
`git commit -m "message"`	Record the staged content in local history.
`git push`	Send local commits to the configured remote branch.
`git pull --ff-only`	Fetch remote commits and advance the local branch if history permits.

Before committing: `git status`, `git diff`, and `git diff --staged`.

Replace placeholders. Push and pull need a configured remote; we connect it in Part II.

Git Snapshots and History



Our inspect-data Skill: Five Checkpoints

Open the inspect-data skill you created in Lecture 4.

skills/inspect-data/SKILL.md

```
---
name: inspect-data
description: Inspect data before analysis.
---

1. Explain rows, keys, variables and units.
2. Check dates, frequency and coverage.
3. Check missing data and duplicate keys.
4. Investigate unusual values.
5. Summarize counts and examples.
Keep the data unchanged.
```

If the skill is missing

```
Create skills/inspect-data/SKILL.md
using the content below.
Create missing folders.
Keep any existing skill unchanged.
Do not inspect the data yet.
[Paste the code on the left.]
```

Copy the prompt and the full code into the same agent message.

Already have the skill? Use your current version.

We will record a copy in Git. Keep your original skill and Treasury CSV unchanged.

Practice 1: The Baseline Commit

YOUR TURN

```
Copy my skill to git-skill-lab/skills/inspect-data/SKILL.md.
Use a new folder outside other repos. Keep the original and CSV.
Add README.md and .gitignore (data/, .env, and editor files).
Check Git and my author name/email. Initialize main if needed.
Stage only README.md, .gitignore, and skills/inspect-data/SKILL.md.
Show the staged file list and diff. Wait for my review.
```

After checking the three files:

```
Commit the reviewed changes as "Add inspection skill baseline".
Show the commit ID and its file list.
```

Check: one commit, three files, and no CSV in the commit.

A Meaningful Change to the Skill

A short missing-data rule can leave the report format open.

Removed line

```
3. Check missing data and duplicate keys.
```

Replacement

For every column, report the missing count and percentage.
Check duplicate keys separately.

The diff shows what changed. Rerunning the skill checks the result.

Already explicit in your skill? Choose another missing check.

Practice 2: A Tested Revision

YOUR TURN

```
Read skills/inspect-data/SKILL.md. Update its missing-data rule
to require a count and percentage for every column.
Keep duplicate-key checks.
Rerun the skill on my Treasury CSV and keep the data unchanged.
Show the new findings and the Git diff. Do not commit yet.
```

Verify one count and its denominator. Inspect the edited lines.

```
Stage only skills/inspect-data/SKILL.md. Show the staged diff
and file list. After my review, commit as
"Clarify missing-value reporting". Show the commit.
```

Check: two commits. The second changes only the skill.

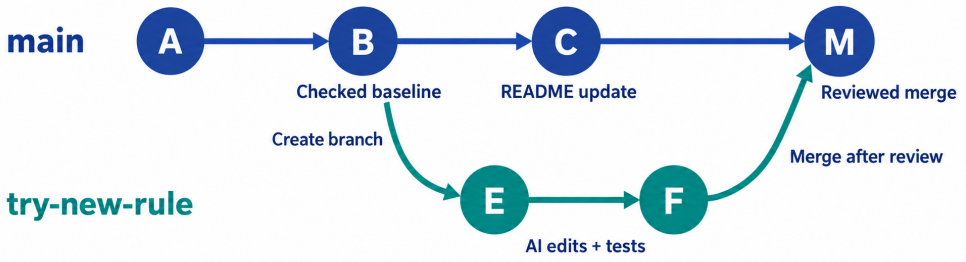
Reading the Earlier Version

Each commit gives you a reference point for the method.

Command	Question it answers
`git log --oneline`	Which versions have I recorded?
`git show <ID>`	What changed in this commit?
`git show <ID>:<path>`	What did this file contain at that commit?

```
Show the baseline version of skills/inspect-data/SKILL.md.
Compare its missing-data rule with the latest version.
Use the actual commit IDs. Leave working files unchanged.
```

2. Branches



A branch tracks a line of commits in the same repository.

Commit or preserve unfinished edits before switching branches.

Creating, Reviewing, and Merging a Branch

Demo: a new reporting rule is an experiment until it passes review.

Create an experiment from clean main

```
git switch -c try-new-rule
```

Ask the agent to edit the rule. Inspect the diff, run checks, then commit on this branch.

`main` keeps its own history until you merge.

Integrate an accepted experiment

```
git switch main
git merge try-new-rule
```

Inspect the combined result and rerun checks. If conflicts appear, resolve and review them first.

Check status before switching: uncommitted edits can carry across branches.

3. Why Git Matters in AI-Assisted Work

An AI-assisted workflow	How you keep track
The agent edits several files quickly.	Review the diff and stage only intended changes.
You want to try another approach.	Use a branch and compare it with a checked baseline.
A later edit makes the result worse.	Identify the responsible commit and reverse that change.

Before the next AI edit, commit a version you have checked. After the edit, inspect the diff and verify the result.

Git records what changed. You still judge whether the analysis is correct.

Recovery After a Bad Change

Example: an AI edit removes a required date check, and the change is committed.

Version	What the skill contains
Earlier version	The date check is present.
Mistaken commit	The date check has been removed.
Recovery commit	The deleted date check is restored.

`git revert` creates a new commit that reverses a selected commit's changes.

The mistaken commit remains in the history, together with the record of its reversal.

Which Files Belong in Git?

A project folder can contain both shared files and files kept only on your computer.

Include in Git

Code and skills: the method you use.

README: how to run the project and obtain its data.

.gitignore: shared rules for excluding files.

Include outputs when sharing them is permitted and useful.

Keep local

Restricted data: provide access instructions instead.

Private configuration: passwords or API keys.

Temporary files: system metadata and notebook checkpoints.

.gitignore is a plain-text file listing patterns for files and folders Git should leave untracked.

.gitignore: Rules and Effects

Save it as git-skill-lab/.gitignore. Write one pattern per line.

Line in .gitignore	What it matches
`data/`	Folders named data and their contents.
`.env`	Local configuration files that may contain API keys.
`.DS_Store`	macOS folder metadata files.
`.ipynb_checkpoints/`	Jupyter's automatic checkpoint folders.

Example: a new data/practice.csv file

It stays on your computer. By default, `git add .` skips it, so it stays out of the next commit and push.

Commit `.gitignore` itself so collaborators receive the same rules.

Already tracked files stay tracked. Adding a rule does not erase earlier commits.

Part II: GitHub

A website for hosting Git repositories and collaborating

Git provides the version history

Commits, branches, comparisons, and merges.

GitHub provides an online workspace

Hosted repositories, access controls, issues, and pull requests.

We will follow one repository through a live website demonstration. Learn what each page and action means.

Accounts, Repositories, and Access

A repository URL identifies its owner and project.

```
github.com/<owner>/<repository>
```

Account / owner	The person or organization that owns the repository.
Repository	Project files and Git history, with related collaboration tools.
Public	Anyone can view it. Write access still requires permission.
Private	Access is limited to the owner and authorized people.

On the website: avatar username, owner/project, and visibility label.

CC Switch: How We Installed It

farion1231/cc-switch: a public project with an MIT license.

In Environment Setup, we installed the packaged app:

Releases → Assets → Download installer → Run installer

macOS: `.dmg` Windows: `.msi`

The same GitHub project also lets us explore and contribute:

README	Understand the app and follow its installation instructions.
Code / Commits	Inspect source files and the history of changes.
Issues	Report a problem or suggest an improvement.
Pull requests	Propose specific changes to the project files.

Install gives us a working app. **Clone** gives us source files and Git history.

Four Ways to Use the CC Switch Repository

What do you want to do with this project?

Star	Save CC Switch so you can find it again.
Fork	Create `yourname/cc-switch` on GitHub, linked to the original project.
Clone	Download a repository with its Git history to work on your computer.
Download ZIP	Download source files for a selected version, without Git history.

A **branch** is a line of development within a repository. A **fork** is another repository. A **clone** is a local copy.

To install the app, follow the README's installation instructions or use **Releases**.

CC Switch: A Kimi Model-Update Issue

Classroom example: a new Kimi model becomes available.

The friction we encountered

We had to manually change the configured model ID to use the new model.

Issue: describe the steps, current behavior, and desired improvement.

A proposed improvement

Let users check or refresh the supported model list, then confirm which model to use.

Keep the current selection until the user chooses to switch.

Suggested issue title: “Make it easier to update Kimi model configuration.”

An **Issue** describes the need. A **PR** proposes the file changes that address it.

From the Kimi Idea to a Pull Request

Suppose you implement the improvement. Where do the changes go?



1. **Fork**, then **clone your fork** to your computer.
2. Create the branch, edit and test, **commit**, then **push to your fork**.
3. Open a **PR**: your branch → the original project's `main`.
4. Maintainers review **Files changed**; they may request edits or merge it.

Pushing to your fork does not change the original project's main branch.

After Merge: Source Code vs. Installed App

Suppose the Kimi improvement is accepted into CC Switch's main.

You work on the source

The original repository now contains the change. Your local clone may still be older.

In a clean clone of the original, on `main`:

```
git status
git pull --ff-only
```

Assumes main tracks the original repository's main.

You use the installed app

Your installed CC Switch does not change just because the PR was merged.

Wait for a release containing the improvement, then update the app through the documented installation method.

PR / merge: accept source changes. **git pull**: update local source.

Release / install: deliver the changed software to app users.

Demo: Ask the Agent to Publish Our Skill

Open the existing git-skill-lab folder from Practice 1 and 2.

Give the coding agent this prompt:

```
Publish this existing git-skill-lab repository to GitHub.
Check the folder, GitHub account, branch, remote and commits.
Keep existing files and history; exclude data and credentials.
Create a private git-skill-lab repo in my account if needed.
Connect the remote and push the existing commits.
Guide me through GitHub sign-in if required.
Return the repository URL and the latest pushed commit ID.
```

On GitHub: verify Private, open README and `SKILL.md`, and check that both practice commits are visible.

Local commits $\xrightarrow{\texttt{git push}}$ **GitHub repository**

Collaborators and Invitations

Owner

Invite the exact GitHub username through the repository's access settings.

Check whether the invitation is pending or accepted.

Invited person

Sign in to the invited account, accept the invitation, then open the repository.

An email notification can arrive in Gmail. Check the GitHub username too.

The repository stays under the owner's name. A collaborator on a personal repository can also write to it.

On the website: Settings, Collaborators, Add people, invitation status.

Share Your Individual Project for Grading

Give the teaching team access to a complete, reproducible project.

1. Use a **private** `Name_StudentID` repository. Push your project with meaningful commit history.
2. Include the **report, code, tables and figures, README, skill, and AI-use disclosure**. Follow the project brief's folder structure.
3. In **Settings** → **Collaborators**, invite `xiongyan2224` and `Jeff7c`. Check invitation status and keep access available.
4. Email the **repository URL** to both:

   `xiongyan2224@gmail.com` and `7chenjeff7@gmail.com`.

Ready for grading: we can open and clone the repository, read the report, and follow the README to reproduce the main tables and figures.

For restricted data, provide access and input-folder instructions instead of uploading the data.